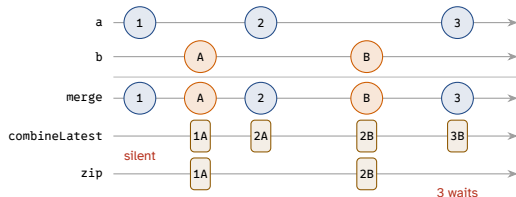


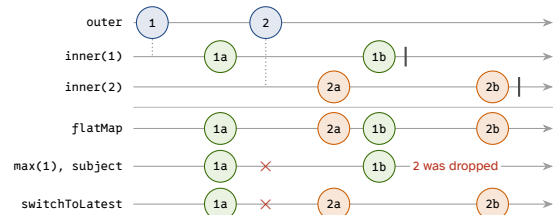
In one line: Every operator is a **struct** deciding **which values** it emits, **how it passes demand** upstream and **when it ends**. One **failure ends the whole chain** unless caught *inside* a flatMap; share() gives one upstream many subscribers. Migrating keeps Combine at the edges: .values / Future.value in, a cancellable Task for the token.

Combining two publishers (time →)



merge: Interleave (same type), ends when *all* end. **combineLatest:** silent until *each* has emitted, then latest of each; never finishes if one never emits. **zip:** pairs the oldest unconsumed of each; ends when *either* ends. Any upstream failure fails the result.

Flattening: each outer value starts an inner publisher



flatMap (.unlimited): all inner live, outputs interleave. **maxPublishers:** .max(1): requests 1 outer value, the next only when the inner ends; a subject's 2 arrives at demand 0 and is **dropped** (from a cold or buffered upstream it waits: 1a 1b 2a 2b, concat). **switchToLatest:** 2 **cancels** inner(1).

How it works

- The chain's type is the nesting: Publishers.Map<Publishers.Filter<...>, T>. At an API boundary hide it: eraseToAnyPublisher() (a box, dynamic dispatch) or some Publisher<Feed, Never> (Publisher has primary associated types, Swift 5.7).
- flatMap needs Self.Failure == P.Failure: align with setFailureType(to:) or mapError. An inner *finishing* does not end the stream; an inner *failing* does (Apple docs).
- try-operators (tryMap, tryFilter, tryCatch...) widen Failure to any Error; mapError gets your type back.
- Errors:** catch = swap in a fallback *publisher*; replaceError(with:) = a value; retry(n) **re-subscribes** upstream up to n times (cold: the request runs again, no delay); assertNoFailure() traps.

Catalogue — by job

transform	map tryMap compactMap scan reduce collect(n) removeDuplicates replaceNil
limit	filter first(where:) prefix(n) prefix(untilOutputFrom:) dropFirst drop(untilOutputFrom:) output(at:)
combine	merge(with:) (<=8; MergeMany) · combineLatest (<=4) · zip (<=4) · append · prepend
flatten	flatMap(maxPublishers:) · switchToLatest
time	debounce throttle delay timeout measureInterval
errors	catch tryCatch retry replaceError mapError setFailureType assertNoFailure
share	share multicast makeConnectable autoconnect
debug	print handleEvents breakpoint breakpointOnError

Example — refresh: one load at a time, errors stay inside

```
refreshTaps
.flatMap(maxPublishers: .max(1)) { _ in // PassthroughSubject<Void, Never>
    api.feedPublisher() // taps while loading: dropped
    .retry(2) // AnyPublisher<Feed, APIError>
    .map(State.loaded) // GET re-sent up to 2 more times
    .catch { Just(State.failed($0)) } // INSIDE: outer stream lives on
}
.receive(on: DispatchQueue.main)
.assign(to: &$state) // no token, no self capture
```

Sharing

- Cold publishers run **per subscriber**: two sinks on one dataTaskPublisher = two requests.
- share() = multicast + PassthroughSubject + implicit autoconnect() (docs); Share is a **class**. No replay: a late subscriber misses values; on a synchronous upstream (Just, an array) the first subscriber drains it all.
- multicast(subject: CurrentValueSubject(...)) replays the latest; makeConnectable() + connect() starts work after all subscribed. Timer.publish(every:on:in:) is connectable: no .autoconnect() = no ticks.

Combine → Swift concurrency

Combine	async/await
Future, one-shot	async func · try await future.value (iOS 15)
many values	for await v in pub.values (iOS 15)
AnyCancellable	a Task you cancel(); SwiftUI .task
receive(on: .main)	@MainActor
PassthroughSubject	AsyncStream — one consumer (loops split)
share()	nothing built in: one stream per consumer
@Published	@Observable (iOS 17) · Observations (iOS 26)
switchToLatest	cancel the old Task · .task(id:)
flatMap(...max(1))	await inside for await (serial)
debounce merge zip	swift-async-algorithms package
retry(n) · scheduler	loop + Task.sleep backoff · inject a Clock

Example — typeahead, the async way

```
// Combine: $query.debounce(...).map(search).switchToLatest()
TextField("Search", text: $query)
.task(id: query) { // new id: old task cancelled
    try? await Task.sleep(for: .milliseconds(300)) // = debounce
    guard !Task.isCancelled else { return } // = switchToLatest
    results = (try? await api.search(query)) ?? [] // main actor
}
```

Testing Combine

- sink into an array, keep the token, fulfil an expectation on completion (.collect() = all at once) — or for await over .values: **start iterating before you trigger**, a subject's earlier value is dropped.
- Inject the scheduler; a test scheduler you advance(by:) makes debounce instant. Cancel: handleEvents(receiveCancel:).

Traps

- !** retry on a POST re-sends it; on a hot subject it only re-subscribes — nothing is replayed.
- !** flatMap(maxPublishers: .max(1)) behind a subject *drops*, not queues — add buffer.
- !** combineLatest stays silent while one input never emitted: prepend(initial).
- !** Deferred { Future { Task { ... } } }: cancelling the subscription does **not** cancel the Task. Future alone runs at creation, even with no subscriber.

Remember

Values, demand, ending — ask all three of every operator. Catch inside the flatMap; share = subject + autoconnect; migrate at the edges: .values in, Task for the token.